

6■Day AWS Mock Interview System – Simplified Development Checklist

Goal: Build a demo■ready mock interview platform with AWS integrations (S3, DynamoDB, Transcribe, Polly, CloudWatch) within 6 days using a small team.

Day 0 – Project Setup

- Create backend repository using Node.js + Express.
- Project structure: routes, controllers, services, middleware, config, utils.
- Configure AWS account (single region recommended: ap-south-1).
- Create IAM user and set environment variables for credentials.
- Install dependencies: express, cors, dotenv, jsonwebtoken, bcrypt, multer, aws-sdk.

Day 1 – Backend Skeleton

- Initialize Express server with modular routing.
- Implement authentication endpoints: /auth/signup, /auth/login, /auth/profile.
- Create interview endpoints: /interview/start, /interview/answer, /interview/history.
- Create resume endpoint: /resume/upload.
- Create analytics endpoint: /analytics/performance.
- Use JWT authentication and consistent API response format.

Day 2 – Storage Integration

- Create Amazon S3 bucket with folders: /resumes and /audio.
- Implement pre-signed S3 upload URLs for files.
- Create DynamoDB tables: Users, InterviewSessions, InterviewAnswers, ResumeMetadata.
- Store user accounts, interview sessions, and answers in DynamoDB.
- Ensure all uploads and records are stored correctly.

Day 3 – Core AI Logic (Simple Version)

- Create question bank JSON grouped by role (e.g., backend, frontend).
- Implement question service to return next question based on role/difficulty.
- Create answer evaluation service with scoring rubric: relevance, correctness, clarity, communication.
- Implement resume analyzer using keyword skill extraction.
- Return detected skills and missing skills compared to job role template.

Day 4 – Speech Processing

- Record interview answers as audio and upload to S3.
- Use Amazon Transcribe to convert audio answers into text.
- Store transcripts and pass them to evaluation service.
- Use Amazon Polly to generate spoken versions of interview questions.
- Store generated audio files in S3.

Day 5 – Analytics

- Compute session score averages.
- Identify strong and weak skill areas.
- Generate improvement suggestions.
- Create analytics API returning session history and performance trends.

Day 6 – Deployment & Monitoring

- Enable CloudWatch logging for API calls and AWS service activity.
- Deploy backend (recommended: EC2 or Elastic Beanstalk for speed).
- Test full flow: signup → interview → answer → scoring → resume upload → analytics.
- Ensure logs appear correctly in CloudWatch.

Final System Test Checklist

- User signup and login working.
- Interview session creation working.
- Questions fetched from question bank.
- Text answer submission evaluated and stored.
- Audio answers transcribed successfully.
- Resume upload and skill extraction working.
- Performance analytics displayed.
- AWS logs visible in CloudWatch.

Key Speed-Up Strategies

- Use rule-based scoring instead of complex ML models.
- Use JSON question bank instead of AI-generated questions.
- Use pre-signed S3 uploads to avoid backend file transfer overhead.
- Use asynchronous speech processing instead of real-time streaming.
- Maintain single backend service instead of microservices.